FlashDrop: Profile Image Fix & Data Persistence

Ready-to-copy implementation prompt for Google AI Studio / your existing Android project

How to use

Copy the prompt below into your existing FlashDrop project. Ask the coding assistant to inspect the current implementation, make the changes, and test them. The prompt distinguishes device-only saving from cloud saving so persistence is not falsely promised.

TASK: Fix FlashDrop profile image display and implement reliable data persistence across the existing app.

PROJECT CONTEXT

- FlashDrop is an Android app built with Kotlin and Jetpack Compose.

- The Profile & Settings screen shows a customer profile card with a circular profile photo/avatar, name, account type, email, and address.

- Currently, after selecting or adding a profile image, the image does not appear correctly on the screen. The app must also save all user-entered and app data reliably.

- Work in the existing project. Do not redesign unrelated screens, remove features, or create a separate demo app.

PART 1 — FIX PROFILE IMAGE DISPLAY

1. Inspect the current profile screen, image picker/camera implementation, image model, and state management. Identify and fix the actual root cause rather than hiding the issue.

2. Support selecting an image from the device gallery and taking a photo with the camera, using the correct Android permissions and Activity Result APIs for the project's Android version.

3. After the user selects or captures a photo, show it immediately in the circular avatar on the Profile & Settings screen and in any profile-edit screen.

4. Use a proper image-loading solution already present in the project (or add a compatible Compose image loader such as Coil if needed). Correctly handle content URIs, file URIs, remote URLs, and loading/error states.

5. Do not keep only a temporary content URI that may stop working later. Copy the chosen image into app-managed persistent storage, or upload it to the existing cloud storage service and save its stable download URL. If using Android's photo picker, persist access where supported or copy the file into app storage.

6. Keep the image aspect ratio correct, crop it to a circle, and avoid stretching, blank images, broken-image icons, or crashes. Show a default avatar only when no valid photo exists.

7. When a photo is changed, update the UI immediately and save the new reference. When the app is reopened, load and display the same saved photo.

8. Handle cancellation, unsupported files, permission denial, missing files, upload failure, and slow loading gracefully. Show a clear message and retain the previous valid photo if saving fails.

9. Make sure the profile image does not cover or push the name, account type, email, address, or edit button off-screen. Keep the layout responsive on different screen sizes.

PART 2 — SAVE ALL FLASHDROP DATA

Persist all relevant app data, not just the profile image, including:

- Customer profiles and profile photos.

- Shopkeeper profiles, shop details, shop photos, product images, and product information.

- Admin and Main Admin settings, registration-mode ON/OFF state, shop approval/verification status, and other configuration.

- Categories, including the "Printing Press" category and its shops.

- Cart contents, quantities, saved addresses, and checkout details.

- Orders, order items, totals, delivery charges, payment method/status, and delivery tracking/status history.

- Any other existing user-created records or settings in the app.

IMPLEMENTATION REQUIREMENTS

1. First inspect the existing architecture and backend. Reuse the current database, repository, authentication, and storage setup wherever possible. Do not replace working services or hard-code sample data.

2. If the app already uses Firebase, use Firebase Authentication for identity, Cloud Firestore (or the project's existing database) for records, and Firebase Storage for profile/shop/product images. Apply proper security rules so users can only edit permitted records and authorized admins can manage admin-only settings.

3. If no backend is configured, implement a clear persistence layer appropriate to the current project. For device-only/demo persistence, use Room for structured records, DataStore for small preferences, and app-internal files for images. Keep repositories abstracted so cloud sync can be added without rewriting screens.

4. Important: device-local storage alone does NOT preserve data after uninstall or guarantee access on another device. If data must survive uninstall/reinstall or be shared between devices, store it in a properly configured cloud backend and associate it with the authenticated account. Do not claim cloud persistence is working unless the backend is actually configured and tested.

5. Save changes at the point they are made and load saved data when the app starts or a screen opens. Use ViewModel/repository state rather than temporary UI-only variables.

6. Keep data consistent across screens. For example, a profile edit must update the profile card; a product edit must update the shop and category listing; an order status change must update order history and tracking.

7. Do not lose existing data during migration. Add safe migrations if database schemas change. Handle network errors, retries, offline states, duplicate submissions, and loading/error feedback.

8. Do not store passwords, Aadhaar numbers, PAN numbers, GST documents, access tokens, or other sensitive credentials in plain text. Use the project's secure authentication and access-control mechanisms. Never expose private documents or profile images publicly without authorization.

9. Ensure users can access only their own data unless their role and permissions explicitly allow more. Validate roles and permissions on the backend, not only by hiding UI buttons.

PART 3 — TEST AND VERIFY

Test and fix the implementation for these scenarios:

- Select a gallery image; it appears immediately and remains after navigating away and returning.

- Capture a photo with the camera; it appears immediately and remains after restarting the app.

- Edit the profile, save it, close/reopen the app, and confirm the name, email (where editable), address, and photo are restored.

- Select a new photo, then cancel another selection; the saved photo remains correct.

- Test missing/unreadable image files and denied permissions without crashing.

- Create/edit a product and its image; verify the product persists after restart.

- Add items to the cart, change quantities, save an address, and verify the expected persistence.

- Create an order and verify its items, totals, delivery charge, and status are retained.

- Change an admin setting and verify it persists and is available only to authorized admins.

- If cloud storage is configured, test sign-in on a second install/device and verify account-linked data syncs correctly.

DELIVERABLES

- Implement the actual fixes in the existing project, not just a mockup or explanation.

- Keep the existing FlashDrop visual design and green/orange branding unless a small change is necessary to fix layout.

- Provide a concise summary of files changed, the root cause of the image issue, the storage approach used, any Firebase/backend configuration still required, and the tests performed.

- Report honestly which data is stored locally and which is stored in the cloud. Do not mark a feature complete if it is only a visual demo.

- Build/run the app and resolve compile errors introduced by these changes.